COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 031 - Visualizations and Data Analysis | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | October 17, 2026 | **DATE SUBMITTED:** | Oct 24, 2026 |
| **NAME:** | Canoy, Hail B.<br>Corro, Yazinel<br>De Leon, Benj Federic | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 13** **Designing a Purpose-Driven Data Dashboard**

## 1. PROCEDURE OUTPUT

### TASK 1

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(10)
students = [f"Student {i}" for i in range(1, 21)]
subjects = ["Math", "Science", "English", "Programming", "Data Analytics"]

data = {
    "Student": np.random.choice(students, 100),
    "Subject": np.random.choice(subjects, 100),
    "Score": np.random.randint(60, 100, 100),
    "Attendance (%)": np.random.randint(70, 100, 100)
}
df = pd.DataFrame(data)

# User Persona 1: TEACHER
PASSING_SCORE = 75
df_teacher = df.copy()
df_teacher["Status"] = df_teacher["Score"].apply(lambda x: "At-Risk (<75)" if x < PASSING_SCORE else "On-Track")

fig_teacher = px.scatter(
    df_teacher,
    x="Attendance (%)",
    y="Score",
    color="Status",
    color_discrete_map={"At-Risk (<75)": "red", "On-Track": "teal"},
    hover_data=["Student", "Subject"],
    title="<b>Teacher View:</b> Student Performance & Risk Identification"
)
fig_teacher.add_hline(y=PASSING_SCORE, line_dash="dash", line_color="red", annotation_text="Threshold (75)")
fig_teacher.show()

# User Persona 2: STUDENT
target_student = "Student 1"
```

```
# User Persona 2: STUDENT
target_student = "Student 1"

# Subject averages across the whole class
class_avg = df.groupby("Subject", as_index=False)["Score"].mean().rename(columns={"Score": "Class Average"})
# Student specific scores
student_data = df[df["Student"] == target_student].groupby("Subject", as_index=False)["Score"].mean().rename(columns={"Score": "My Average"})

# Merge metrics
student_comparison = pd.merge(class_avg, student_data, on="Subject", how="left")

fig_student = go.Figure()
fig_student.add_trace(go.Bar(
    x=student_comparison["Subject"],
    y=student_comparison["Class Average"],
    name="Class Average",
    marker_color="lightslategrey"
))
fig_student.add_trace(go.Bar(
    x=student_comparison["Subject"],
    y=student_comparison["My Average"],
    name=f"{target_student}'s Average",
    marker_color="royalblue"
))

fig_student.update_layout(
    barmode="group",
    title=f"<b>Student View:</b> Personal Progress vs. Class Average ({target_student})",
    xaxis_title="Subject",
    yaxis_title="Score"
)
fig_student.show()

# User Persona 3: DEPARTMENT HEAD
subject_summary = df.groupby("Subject", as_index=False).agg(
    Average_Score=("Score", "mean"),
    Average_Attendance=("Attendance (%)", "mean")
)
```

```
)
fig_student.show()

# User Persona 3: DEPARTMENT HEAD
subject_summary = df.groupby("Subject", as_index=False).agg(
    Average_Score=("Score", "mean"),
    Average_Attendance=("Attendance (%)", "mean")
)

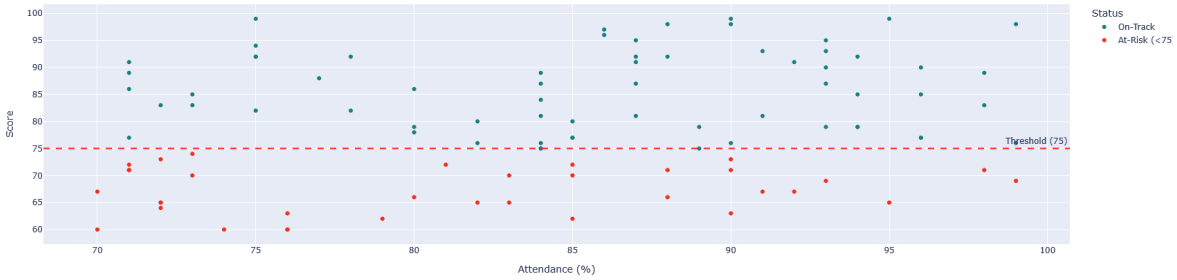
fig_dept = make_subplots(
    rows=1, cols=2,
    subplot_titles=("Average Score by Subject", "Average Attendance (%) by Subject")
)

fig_dept.add_trace(
    go.Bar(x=subject_summary["Subject"], y=subject_summary["Average_Score"], name="Avg Score", marker_color="indigo"),
    row=1, col=1
)

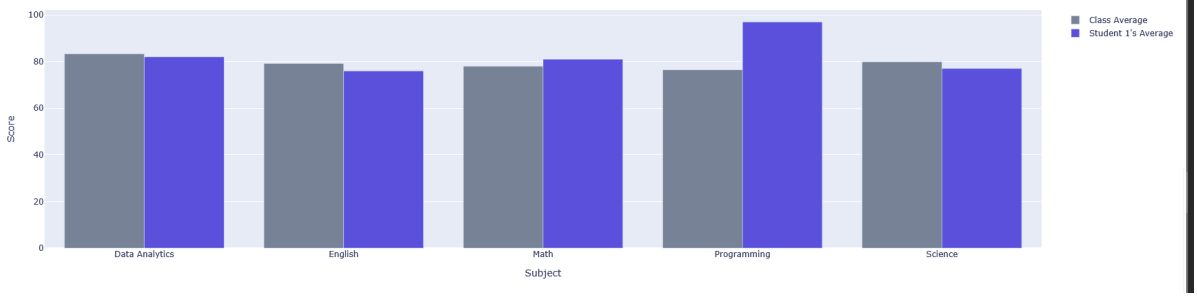
fig_dept.add_trace(
    go.Bar(x=subject_summary["Subject"], y=subject_summary["Average_Attendance"], name="Avg Attendance", marker_color="darkcyan"),
    row=1, col=2
)

fig_dept.update_layout(
    title_text="<b>Department Head View:</b> Subject-Level Summary Performance Dashboard",
    height=450,
    showlegend=False
)
fig_dept.show()
```

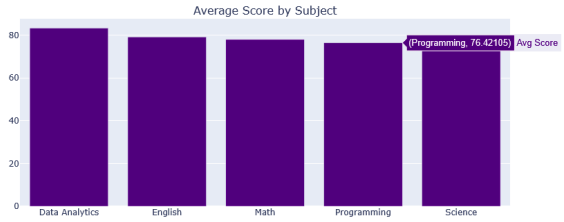
**Teacher View:** Student Performance & Risk Identification

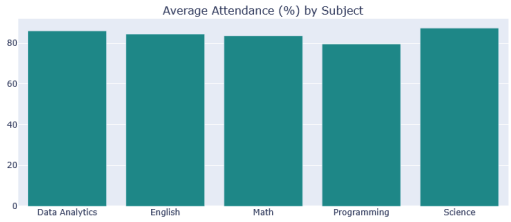



**Student View:** Personal Progress vs. Class Average (Student 1)




**Department Head View:** Subject-Level Summary Performance Dashboard



## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

```
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# 1. Simulate Hourly Energy Consumption Data
np.random.seed(42)
hours = [f"{h:02d}:00" for h in range(24)]
campus_areas = ["Laboratories", "Offices", "Classrooms", "Library", "Dormitories"]

data_rows = []
for area in campus_areas:
    for h_idx, hour in enumerate(hours):
        base = 50 if area in ["Laboratories", "Dormitories"] else 20
        multiplier = 2.5 if 8 <= h_idx <= 18 else 1.0
        consumption_kwh = round(base * multiplier + np.random.uniform(5, 15), 2)

        data_rows.append({
            "Hour": hour,
            "Hour_Num": h_idx,
            "Campus_Area": area,
            "Energy_Consumption_kWh": consumption_kwh
        })

df_energy = pd.DataFrame(data_rows)


total_by_area = df_energy.groupby("Campus_Area", as_index=False)["Energy_Consumption_kWh"].sum()
hourly_total = df_energy.groupby(["Hour", "Hour_Num", "Campus_Area"], as_index=False)["Energy_Consumption_kWh"].sum().sort_values("Hour_Num")
fig = make_subplots(
    rows=2, cols=2,
    specs=[
        [{"type": "xy", "colspan": 2}, None],
        [{"type": "xy"}, {"type": "domain"}]
    ],
    subplot_titles=(
        "Hourly Energy Consumption Trend (Line Chart)",
        "Total Energy Consumption by Area (Bar Chart)",
        "Overall Consumption Proportion (Pie Chart)"
    ),
    vertical_spacing=0.15,
```

```
        vertical_spacing=0.15,
        horizontal_spacing=0.10
    )

    # Chart 1: Line Chart (24-Hour Trend by Area)
    for area in campus_areas:
        area_df = hourly_total[hourly_total["Campus_Area"] == area]
        fig.add_trace(
            go.Scatter(
                x=area_df["Hour"],
                y=area_df["Energy_Consumption_kWh"],
                mode="lines+markers",
                name=area
            ),
            row=1, col=1
        )

    # Chart 2: Bar Chart (Total Consumption per Area)
    fig.add_trace(
        go.Bar(
            x=total_by_area["Campus_Area"],
            y=total_by_area["Energy_Consumption_kWh"],
            marker_color=px.colors.qualitative.Set2[:len(campus_areas)],
            showlegend=False,
            name="Total kWh"
        ),
        row=2, col=1
    )

    # Chart 3: Pie Chart (Proportion of Total Energy Use)
    fig.add_trace(
        go.Pie(
            labels=total_by_area["Campus_Area"],
            values=total_by_area["Energy_Consumption_kWh"],
            hole=0.4,
            marker=dict(colors=px.colors.qualitative.Set2[:len(campus_areas)])
        ),
        row=2, col=2
    )

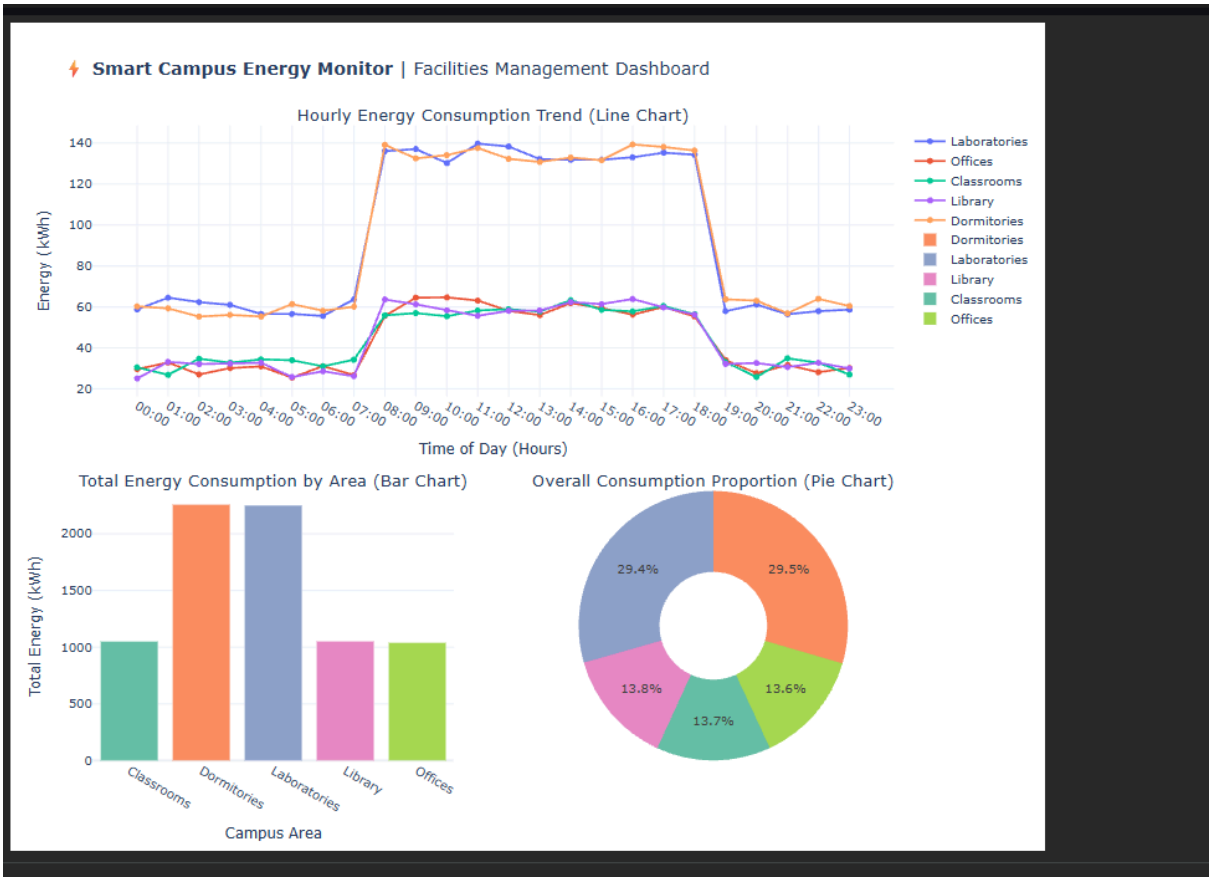
    fig.update_layout(
        height=800,
        width=1000,
        title_text="<b>⚡ Smart Campus Energy Monitor</b> | Facilities Management Dashboard",
        template="plotly_white",
        hovermode="x unified"
    )

    fig.update_xaxes(title_text="Time of Day (Hours)", row=1, col=1)
    fig.update_yaxes(title_text="Energy (kWh)", row=1, col=1)

    fig.update_xaxes(title_text="Campus Area", row=2, col=1)
    fig.update_yaxes(title_text="Total Energy (kWh)", row=2, col=1)

    fig.show()
```

## OUTPUT




Engineering decisions:

A facilities manager can easily view peak usage hours and unusual energy spikes over time by placing the 24-hour line chart at the top of the dashboard layout. Total consumption comparisons and overall percentage splits across campus zones are clearly displayed in the bar and pie charts at the bottom. In order to minimize visual clutter and facilitate cross-chart comparison, all three charts employ the same color scheme for each campus area. Lastly, interactive hover tooltips were added to make it simple to identify and modify facilities by allowing the manager to hover over any specific data point to display precise numbers.

| CONCLUSIONS | |
|---|---|
| Canoy | In this activity, I learned how to create interactive dashboards using Plotly to make data easier to understand. I also learned how to use line, bar, and pie charts to show information clearly and help users find important details from the data. |
| Corro | In conclusion, I learned how to make interactive Plotly dashboards so that users can hover over data points, and dynamically explore details to get useful information. Also, I have experience working with data and creating interactive dashboards with Plotly to tailor to the specific needs of end-users such as teachers, students and department heads. and Developing the supplement taught me how to combine chart types line, bar and pie to clearly communicate operational trends and metric breakdowns. |
| De Leon | Dashboards must fit the person using them. Different people need to see different facts. Clear charts and simple colors make data easy to read. Tools like Plotly let you add menus and highlight key info. This helps people see what matters and make smart choices quickly. |

| COLAB LINKS | |
|---|---|
| Canoy | Hands_On_Activity_13___Designing_a_Purpose_Driven_Data_Dashboard_C… |
| Corro | Hands_On_Activity_13___Designing_a_Purpose_Driven_Data_Dashboard_G… |
| De Leon | Hands_On_Activity_13___Designing_a_Purpose_Driven_Data_Dashboard_DE LEON.ipynb |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.




I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.